

DSA & Advanced DSA Roadmap

Phase 12 — Heap

Learning Objectives

By the end of this phase, learners will be able to:

- Implement a min-heap and a max-heap from scratch backed by an array, including heapify, insert, and extract-min/max.
- Explain why a heap gives $O(\log n)$ insertion and extraction but only $O(1)$ access to the min/max, and why it does not support fast arbitrary search.
- Connect a heap to the abstract priority queue interface, and explain the relationship between heap sort (Phase 8) and heap-based priority queues.
- Identify and apply core heap patterns: top-K selection, K-closest selection, running median maintenance via two heaps, and K-way merging.
- Solve Easy to Medium level heap problems independently, and approach Hard problems (Find Median from Data Stream, Merge k Sorted Lists) using the two-heap and heap-of-iterators techniques respectively.

Implementation & Internals

Core Concepts

- Min Heap — a complete binary tree (stored as an array) where every parent is less than or equal to its children, so the minimum is always at the root.
- Max Heap — the mirror invariant: every parent is greater than or equal to its children, so the maximum is always at the root.
- Heapify — the $O(n)$ bottom-up process of converting an arbitrary array into a valid heap, and the $O(\log n)$ sift-down/sift-up operations used after each insert or extract.
- Priority Queue — the abstract interface (insert, get-min/max, extract-min/max) that a heap implements; revisiting Phase 5's brief mention of Priority Queue Basics, now with a full implementation.

Implementation Exercises

- Build a min-heap from scratch using an array, implementing insert (sift-up) and extract-min (swap-to-root, then sift-down).
- Implement heapify on an arbitrary array in $O(n)$ time, and verify against repeatedly inserting each element into an empty heap ($O(n \log n)$) to confirm both produce a valid heap, only at different costs.
- Implement a max-heap by reusing the min-heap code with negated comparisons, demonstrating the common 'min-heap library, negate values for max-heap behavior' trick used throughout this phase's problems.

Pattern Mastery

Pattern 1: Top K Elements

Concepts

- Maintaining a size-k heap of the best candidates seen so far, pushing new candidates and popping the worst once the heap exceeds size k.
- Choosing min-heap vs max-heap based on which direction needs pruning: a size-k min-heap keeps the k largest elements (the root is the worst of the kept set and gets evicted first).

Practice Problems

- Given an unsorted array and an integer k , find the k th largest element without fully sorting the array.
- Design a class that maintains the k th largest element in a stream of integers, supporting an add operation.

LeetCode

- [Kth Largest Element in an Array](https://leetcode.com/problems/kth-largest-element-in-an-array) — leetcode.com/problems/kth-largest-element-in-an-array
- [Kth Largest Element in a Stream](https://leetcode.com/problems/kth-largest-element-in-a-stream) — leetcode.com/problems/kth-largest-element-in-a-stream

Note: Top K Frequent Elements (the canonical 'top K by frequency' problem) was already assigned in Phase 6 as a hash-map-plus-heap exercise and is not repeated here; this phase's Top K Elements problems instead focus on plain numeric ranking.

Pattern 2: K Closest Elements

Concepts

- A variation of top-K where 'best' is defined by distance rather than raw value, requiring a custom comparator (or transforming each element to its distance) before applying the same size- k heap technique.
- Recognizing when a heap-based approach ($O(n \log k)$) is preferable to sorting everything ($O(n \log n)$), particularly when k is much smaller than n .

Practice Problems

- Given an array of points on a 2D plane and an integer k , find the k points closest to the origin.

LeetCode

- [K Closest Points to Origin](https://leetcode.com/problems/k-closest-points-to-origin) — leetcode.com/problems/k-closest-points-to-origin

Pattern 3: Running Median

Concepts

- Maintaining two heaps simultaneously — a max-heap for the lower half of seen values and a min-heap for the upper half — keeping their sizes balanced so the median is always at one or both roots.
- Rebalancing after every insertion: if one heap grows more than one element larger than the other, move its root into the other heap.

Practice Problems

- Design a data structure that supports adding integers from a stream and finding the median of all elements added so far at any point.

LeetCode

- [Find Median from Data Stream](https://leetcode.com/problems/find-median-from-data-stream) — leetcode.com/problems/find-median-from-data-stream

Pattern 4: Merge K Sorted Lists

Concepts

- Maintaining a heap of 'next candidate' pointers, one per list, always extracting the smallest and advancing only that one list's pointer.
- Generalizing pairwise merging (merge two sorted lists at a time) to a single heap-driven pass that merges all k lists together in $O(N \log k)$ rather than $O(N \cdot k)$.

Practice Problems

- Given k sorted linked lists, merge them into one sorted linked list using a min-heap to always select the next smallest head node.

Note: this exact problem (Merge k Sorted Lists, LeetCode 23) was already assigned in Phase 3 as a deliberate heap preview, with the heap treated as a black box at the time. This phase is where that black box gets opened: revisit the Phase 3 solution now that min-heap internals (heapify, sift-up, sift-down) have been built from scratch, and re-implement it using your own heap rather than a library one.

Phase Assessment

Implementation Tasks

- Build a min-heap from scratch supporting insert, extract-min, and heapify, then derive a max-heap from it using negation.
- Re-implement Phase 3's Merge k Sorted Lists solution using your own from-scratch heap instead of a library priority queue.
- Analyze and explain why the two-heap running-median technique keeps both heaps within size 1 of each other, and what would go wrong if rebalancing were skipped.

Recommended LeetCode Target

Easy Problems

- leetcode.com/problems/kth-largest-element-in-a-stream
- leetcode.com/problems/last-stone-weight

Medium Problems

Top K Elements

- leetcode.com/problems/kth-largest-element-in-an-array

K Closest Elements

- leetcode.com/problems/k-closest-points-to-origin

Hard Problems

- leetcode.com/problems/find-median-from-data-stream

Merge k Sorted Lists (LeetCode 23) — already counted toward Phase 3's Hard tier; revisit here as a re-implementation exercise rather than a new assignment.

On scope and dedup

This phase has 5 newly-assigned problems plus one explicit revisit (Merge k Sorted Lists from Phase 3), smaller than the 7–9 problem range of Phases 4–11. The smaller new-problem count reflects how much of this phase's pattern space was deliberately seeded earlier: Merge k Sorted Lists in Phase 3 and Top K Frequent Elements in Phase 6. None of the 5 newly-assigned problems overlaps with any problem used in Phases 0–11.

Coding Challenges (Assessment Day)

- Kth Largest Element in an Array
- K Closest Points to Origin
- Find Median from Data Stream
- Re-implement Merge k Sorted Lists using a from-scratch heap

Expected Outcome

Students should be able to:

- Implement a min-heap and max-heap from scratch, including heapify and the sift-up/sift-down operations.
- Recognize and apply the four heap patterns covered: top-K selection, K-closest selection, running median, and K-way merge.
- Solve unseen heap problems using pattern-based thinking rather than memorized solutions.
- Explain, with reference to Phase 8 (Sorting), why heap sort and heap-based priority queues share the same underlying data structure but solve different problems.